Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение высшего образования
«Алтайский государственный технический университет им. И. И. Ползунова»

Университетский технологический колледж

Отчет защищен с оценкой_____
Преподаватель_____ *С. В. Умбетов*
«_____»_____2025 г.

Отчёт по практической работе №2 по теме:

«HTTP-сервер на Node.js для статического HTML.»

Студент группы 1ИСП-21 А.О. Яимов
группа и.о., фамилия

Преподаватель *доцент*, к. *т. н.* *С. В. Умбетов*
должность, ученая степень и.о., фамилия

БАРНАУЛ 2025

Практическая работа №2

Задания к лабораторной работе:

1. Написать HTTP сервер на node.js для вывода статичного html файла созданного в lr1.
2. Весь код грамотно разделенный.
3. За отображение файла отвечает node.js
4. Если пользователь обращается к любому адресу кроме индекса то вы ему должны показать 404

Задание принял: Яимов А.О.

Подпись ФИО

Ход работы

В самом начале была взята директория из первой практической работы и переработана иерархия (Рисунок 1)

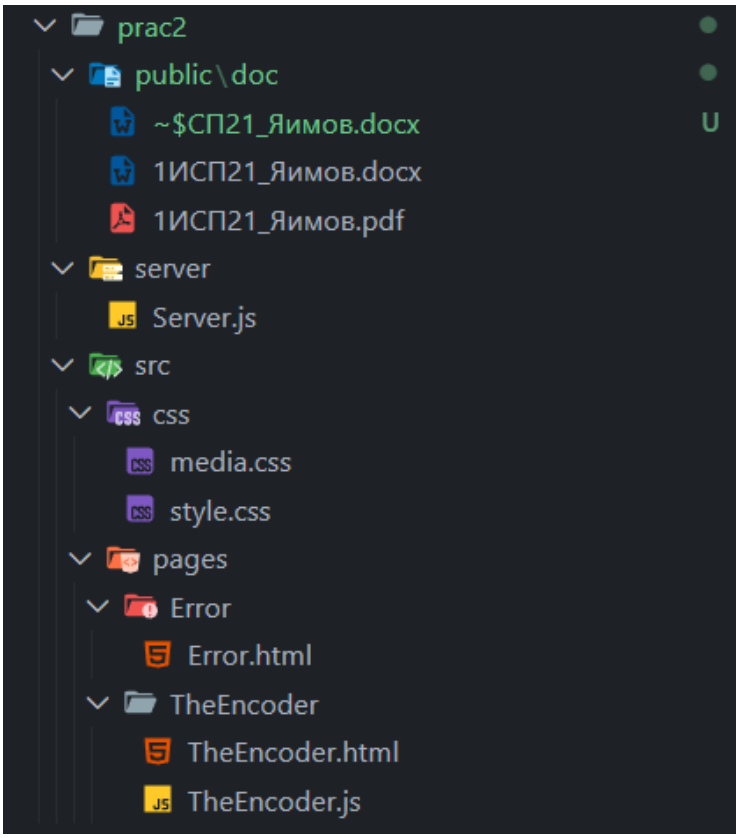


Рисунок 1 – иерархия проекта

Затем была начата разработка серверной части приложения. Был реализован сервер, слушающий 3000 порт, который обрабатывает html с выводом на страницу (Рисунок 2–4)



Рисунок 2 – код сервера

```
C:\Users\yimov\Desktop\STUD\prodpractice2\prac2\server>node Server
listening port 3000
Server request
/ GET
Server request
/favicon.ico GET
```

Рисунок 3 – запуск сервера

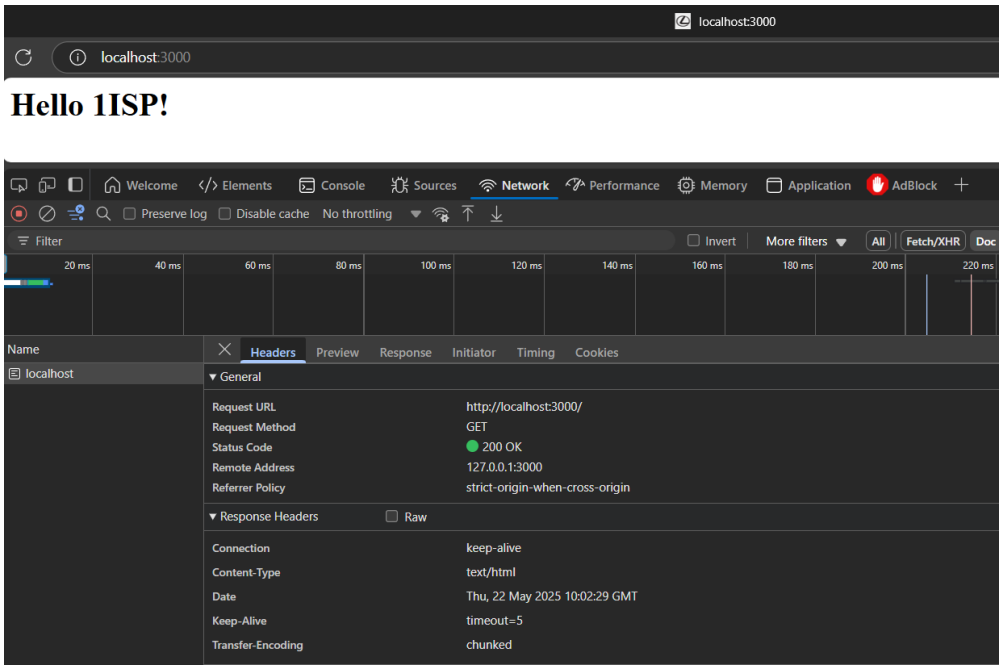


Рисунок 4 – внешний вид

После вносим изменения для вывода html страницы (Рисунок 5), была допущена ошибка в пути файла которую можно увидеть в логах (Рисунок 6)

Server.js M

prac2 > server > Server.js > ...

```
const http = require('http');
const fs = require('fs'); // file system
const path = require('path'); // forming the correct path

const PORT = 3000

const server = http.createServer((req, res) => {
    console.log('Server request');
    console.log(req.url, req.method);

    res.setHeader('Content-type', 'text/html')

    if (req.url = '/') {
        fs.readFile('/public/index.html', (err, data) => {
            if (err) {
                console.log(err)
                res.end();
            }
            else {
                res.write(data);
                res.end();
            }
        })
    }
});

server.listen(3000, 'localhost', (error) => {
    error ? console.log(error) : console.log(`listening port ${PORT}`);
});
```

Рисунок 5 – код сервера

```
C:\Users\yimov\Desktop\STUD\prodpractice2\prac2\server>node Server
listening port 3000
Server request
/ GET
[Error: ENOENT: no such file or directory, open 'C:\public\index.html'] {
  errno: -4058,
  code: 'ENOENT',
  syscall: 'open',
```

Рисунок 6 – логи

После решения ошибки и внесения улучшений (Рисунок 7–8) отобразилась html страница (Рисунок 9), однако CSS и JS не подключились, это связано с тем что сервер в текущей конфигурации обрабатывает только HTML-файлы, но не обслуживает статические файлы (CSS, JS и т д).

Server.js | 44f0429 - commit-9-1-save-dev (2 файлы) | Error.html | style.css | TheEncoder.html

prac2 > server > Server.js > ...

```
const http = require('http');
const fs = require('fs'); // file system
const path = require('path'); // forming the correct path

const PORT = 3000

const server = http.createServer((req, res) => {
    console.log('Server request');
    console.log(req.url, req.method);

    res.setHeader('Content-type', 'text/html')

    const createPath = (page) => path.resolve('src', 'pages', `${page}`, `${page}.html`);

    let basePath = '';
    switch(req.url) {
        case '/':
            basePath = createPath('TheEncoder');
            res.statusCode = 200;
            break;
        default:
            basePath = createPath('Error');
            res.statusCode = 404;
            break;
    }
```

Рисунок 7 – код сервера

```
    fs.readFile(basePath, (err, data) => {
        if (err) {
            console.log(err)
            res.end();
        }
        else {
            res.write(data);
            res.end();
        }
    })

});

server.listen(3000, 'localhost', (error) => {
    error ? console.log(error) : console.log(`listening port ${PORT}`);
});
```

Рисунок 8 – код сервера

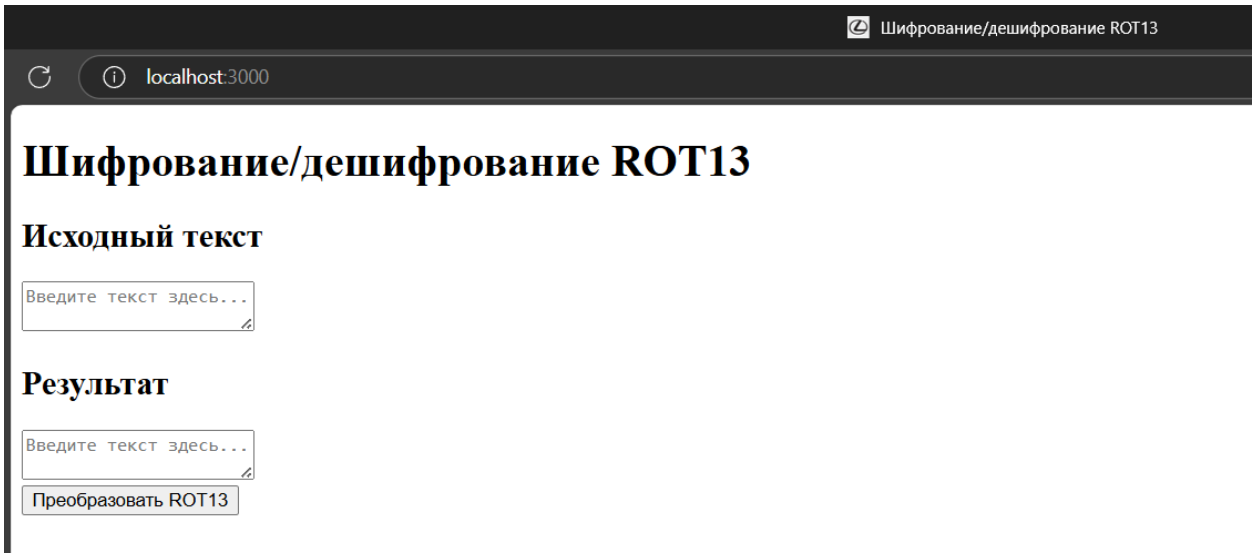


Рисунок 9 – вид страницы

После выяснения причин не отображения CSS, JS внесем улучшения (Рисунок 10–11)

Server.js | e80cd7d - commit-10-add-rounti... (5 файлы) | Error.html | style.css | TheEncoder.html

prac2 > server > Server.js > server > http.createServer() callback > fs.readFile() callback

```
const http = require('http');
const fs = require('fs'); // файловая система
const path = require('path'); // корректная работа с путями

const PORT = 3000;

const server = http.createServer((req, res) => {
    // обработчик запросов
    console.log('Server request');
    console.log(req.url, req.method);

    // расширение файла
    const extname = path.extname(req.url);
    let contentType = 'text/html';
    let basePath = '';

    // устанавливаем Content-Type в зависимости от расширения файла
    switch (extname) {
        case '.css':
            contentType = 'text/css';
            basePath = path.resolve(__dirname, '..', 'src', 'css', path.basename(req.url));
            break;
        case '.js':
            contentType = 'text/javascript';
            basePath = path.resolve(__dirname, '..', 'src', 'pages', 'TheEncoder', path.basename(req.url));
            break;
        default:
            // обработка HTML-страниц
            switch(req.url) {
                case '/':
                    basePath = path.resolve(__dirname, '..', 'src', 'pages', 'TheEncoder', 'TheEncoder.html');
                    break;
                default:
                    basePath = path.resolve(__dirname, '..', 'src', 'pages', 'Error', 'Error.html');
                    break;
            }
    }
```

Рисунок 9 – код сервера

```
    fs.readFile(basePath, (err, data) => {
        if (err) {
            console.log(err);

            // если файл не найден
            if (err.code === 'ENOENT') {
                return sendErrorPage(res, 404);
            }

            // для других ошибок - серверная ошибка
            return sendServerError(res);
        }

        // если файл успешно прочитан отправляем его
        res.writeHead(200, { 'Content-Type': contentType });
        res.end(data);
    });

    // вспомогательные функции
    function sendErrorPage(res, statusCode) {
        const errorPagePath = path.resolve(__dirname, '..', 'src', 'pages', 'Error', 'Error.html');
        fs.readFile(errorPagePath, (err, data) => {
            if (err) return sendServerError(res);
            res.writeHead(statusCode, { 'Content-Type': 'text/html' });
            res.end(data);
        });
    }

    function sendServerError(res) {
        res.writeHead(500);
        res.end('Server Error');
    }
});

server.listen(PORT, 'localhost', (error) => {
    error ? console.log(error) : console.log(`listening port ${PORT}`);
});
```

Рисунок 10 – код сервера

Была реализована обработка JS, CSS и HTML, так же выполнена проверка на ошибку “ENOENT” и сделаны вспомогательные функции для статусов кода 404 и 500. Итоговый вид отображения файлов (Рисунок 11–12).

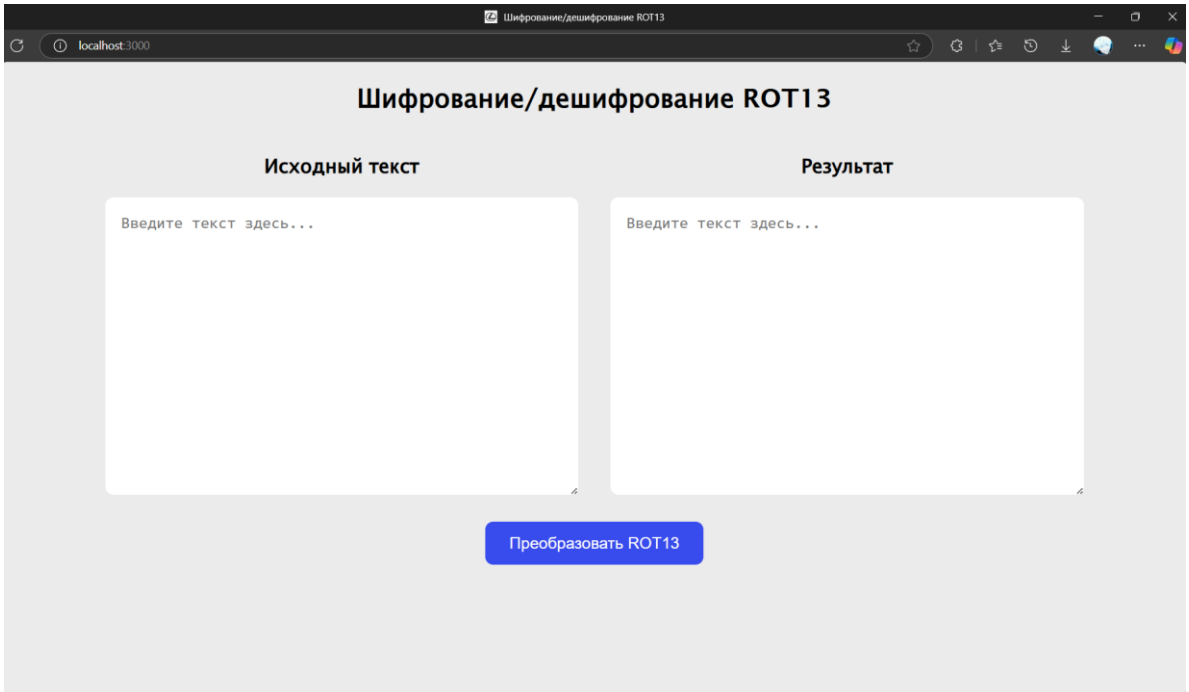


Рисунок 11 – главная страница

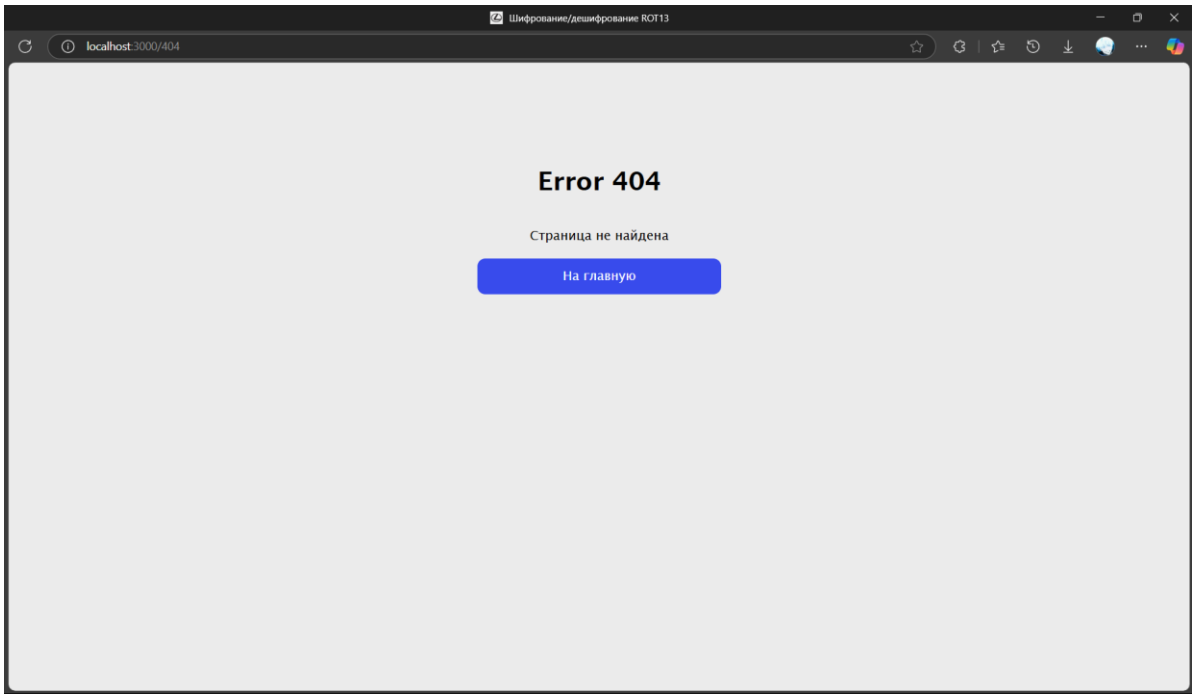


Рисунок 12 – ошибка ENOUT

Итоговый вид коммитов с учетом первой практической работы

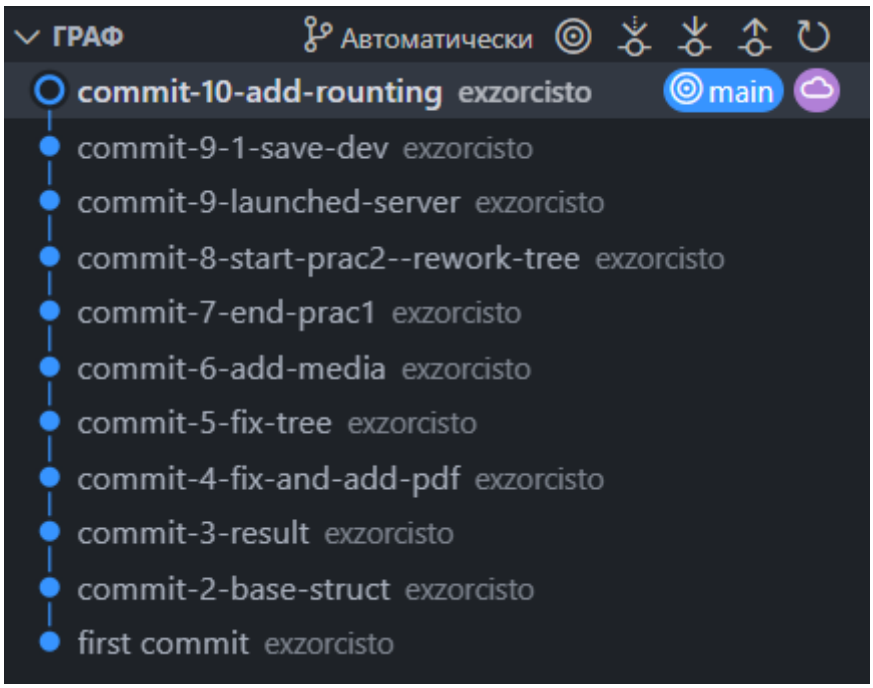


Рисунок 13 – структура коммитов

Вывод

1. **Реализация сервера на Node.js**
   Сервер создан с использованием встроенного модуля http Node.js, что соответствует первому пункту ТЗ. Сервер корректно обрабатывает HTTP-запросы и возвращает статические файлы.

2. **Грамотное разделение кода**
   Код логически структурирован:
   - Четкое разделение на основные блоки (импорт модулей, конфигурация, обработчики)
   - Использование вспомогательных функций (sendErrorPage, sendServerError)
   - Логика определения типа контента вынесена в отдельный switch-case блок

3. **Обработка и отображение файлов средствами Node.js**
   Сервер использует модули fs и path для работы с файловой системой:
   - Чтение файлов происходит асинхронно с помощью fs.readFile
   - Корректное формирование путей с помощью path.resolve и path.extname
   - Установка правильных MIME-типов (Content-Type) для разных типов файлов

4. **Обработка несуществующих маршрутов (404)**
   Реализована полная обработка ошибочных маршрутов:
   - По умолчанию для всех неизвестных URL устанавливается путь к странице ошибки
   - При попытке доступа к несуществующему файлу возвращается статус 404
   - Специальная функция sendErrorPage унифицирует обработку ошибок

Ссылка на репозиторий: exzorcisto/prodpractice2